CO1 – ASSIGNMENT 1

C Programming: Rural Electricity Billing System

Name	B. Janani
Register Number	1912260492
Course	C Programming
Assignment	CO1 – Assignment 1
Case Study	Grama Jyothi Rural Electricity Co-operative

Prepared from the assignment brief supplied for CO1. The source brief contains Tasks 1–11 and only Part A (Impact Analysis) of Task 12.

How to use this document

This PDF combines the written solutions, C source programs, and terminal-style output captures. Each program was compiled with GCC using C11 and warning flags, then executed in this environment. Output captures show the actual stdout from those executions. System-dependent sizes and floating-point output may differ on another compiler or computer.

Important scope note: The uploaded assignment brief ends during Task 12, Part A. Therefore, this document answers the visible Task 12 requirements only and does not invent missing Task 12 sections.

TASK 1 – Understand the Problem and Identify the Data

Inputs and outputs

• **Previous reading and current reading:** meter register values from 0 to 65535. They are non-negative integer readings; a 16-bit register wraps after 65535.

• **Household number:** an identifier, stored as an integer or string depending on its format. If leading zeroes are meaningful, use a character array.

• **Category:** household class or tariff category; an enum or small integer code is suitable.

• **Units consumed:** unsigned integer difference after rollover handling.

• **Energy charge:** calculated from slab rates and stored in paise.

• **Fixed charge:** constant charge in paise.

• **Total bill:** fixed charge plus energy charge, in paise; print as rupees and two paise digits.

Constants, rules and assumptions

• Constants: meter maximum 65535, register modulus 65536, slab limits 100 and 200 units, rates 0/225/450 paise per unit, and fixed charge 5000 paise.

• A valid bill cannot be negative. Total paise must equal fixed charge plus energy charge.

• Consumption must account for a single register rollover between readings. Multiple rollovers cannot be inferred from only two readings.

• Assume readings are from the same meter and period, and the tariff is applied to the measured consumption.

• The example tariff makes the first 100 units free, units 101–200 cost Rs 2.25 each, and units above 200 cost Rs 4.50 each.

Text readings and complexity

If readings are strings such as “01530”, conversion and numeric comparison require scanning up to d digits, so conversion is O(d) time. A fixed-width integer comparison is O(1). A string needs approximately d bytes plus a terminating null character; an unsigned 16-bit reading needs 2 bytes on a system where unsigned short is 2 bytes. Text preserves leading zeroes, which is useful for display, but numeric computation should convert and validate the value first.

TASK 2 – Choose the Data Types

Recommended types: **unsigned short** for the 16-bit meter register; **unsigned int** for per-reading consumption; **long** or **long long** for money in paise; **char arrays** for identifiers that may contain leading zeroes; and **enum** for a small fixed set of categories. Use **double** only for non-financial measurement or reporting where exact paise arithmetic is not required.

Why integer paise is preferred

Binary floating-point cannot represent many decimal fractions exactly. Repeated addition can accumulate small representation errors. Integer paise represents every whole paisa exactly, and rounding is avoided during the billing calculation. Convert to rupees only when printing.

Rollover calculation

Unsigned 16-bit arithmetic wraps modulo 65536. Thus (unsigned short)(120 – 65400) = 256 because 65536 – 65400 + 120 = 256. The result is a positive modular difference, not a negative value. This method assumes no more than one rollover occurred.

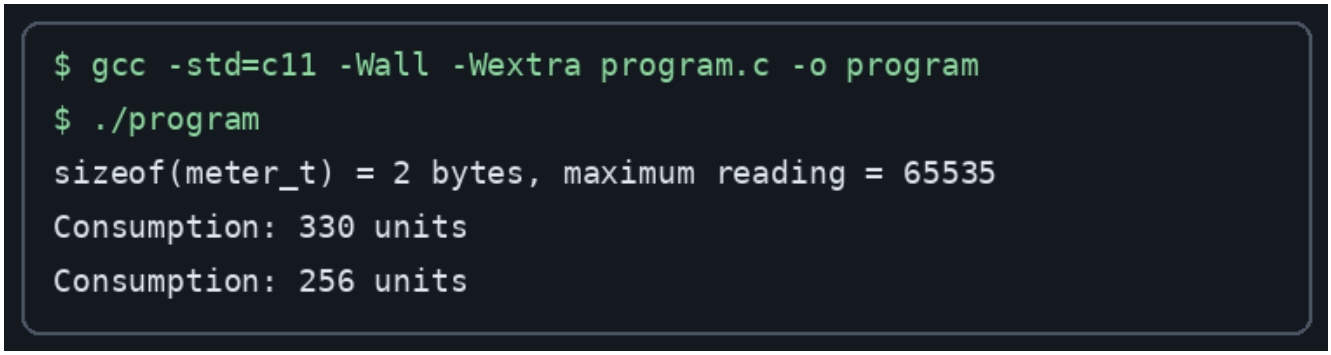
Completed TODO 1 – rollover program

```
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    return (unsigned short)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));
    return 0;
}
```



On a typical system with 2-byte unsigned short, the program reports 330 units for 1200→1530 and 256 units for 65400→120.

Size and range

Use sizeof, limits.h and float.h on the target system rather than assuming every platform has identical sizes. C guarantees minimum ranges, not identical byte sizes. USHRT_MAX is at least 65535; UINT_MAX is at least 65535; LONG_MAX is at least 2147483647; LLONG_MAX is at least 9223372036854775807. The actual values can be printed using the macros.

Example: adding a decimal value such as 0.10 to a float many times may not equal the exact decimal total. Integer paise avoids that issue. For 4,000 households, even a small per-bill floating error can become visible when summed.

TASK 3 – Define the Constants

Use named macros for tariff values shared by the program: SLAB1_UNITS, SLAB2_UNITS, RATE1_PAISE, RATE2_PAISE, RATE3_PAISE and FIXED_CHARGE_PAISE. A tariff revision then changes one definition rather than many calculations.

• **#define:** preprocessor text substitution; no variable storage, but no C type checking and less visibility in a debugger.

• **const:** typed and scoped, visible to the compiler and debugger; it may occupy storage depending on use and optimization.

• **enum:** appropriate for named integral choices such as household categories or states, not decimal tariff values.

• Use uppercase names with underscores for macros and constants.

Completed TODO 1 – slab billing

```
#include <stdio.h>
```

```c
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;
    if (units > SLAB2_UNITS)
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
    if (units > SLAB1_UNITS) {
        int slab2 = units < SLAB2_UNITS ? units - SLAB1_UNITS
                                        : SLAB2_UNITS - SLAB1_UNITS;
        paise += (long)slab2 * RATE2_PAISE;
    }
    return paise + (long)(units < SLAB1_UNITS ? units : SLAB1_UNITS)
                 * RATE1_PAISE;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    for (int i = 0; i < 5; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
    }
    return 0;
}
```

```
$ gcc -std=c11 -Wall -Wextra program.c -o program
$ ./program
0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

The program uses integer paise. Results: 0 and 100 units cost Rs 50.00; 150 units cost Rs 162.50; 200 units cost Rs 275.00; 350 units cost Rs 950.00. Changing RATE2_PAISE from 225 to 230 changes all applicable bills consistently.

TASK 4 – Use Macros Safely

A macro is expanded by the preprocessor before compilation. Fully parenthesise both parameters and the entire expression. For example, SQUARE(x) should be ((x) * (x)); otherwise SQUARE(a+1) can expand to a+1*a+1 and follow operator precedence unexpectedly.

• MAX(a,b) may evaluate one argument more than once. MAX(i++, j) can therefore change i unexpectedly.

• A function or static inline function is safer when arguments have side effects or when type checking matters.

• A macro used k times can duplicate its expanded expression at each use. An inline function may allow code sharing or compiler inlining; actual code size depends on optimization.

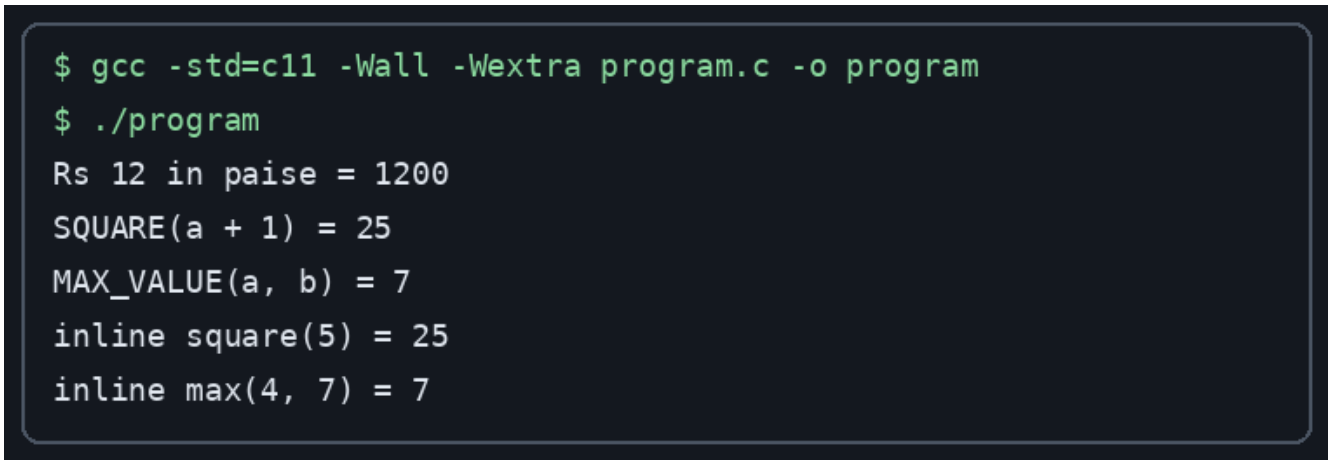
Safe macro and inline function example

```c
#include <stdio.h>
#define RUPEES_TO_PAISE(r) ((long)(r) * 100L)
#define SQUARE(x) ((x) * (x))
#define MAX_VALUE(a, b) ((a) > (b) ? (a) : (b))

static inline int square_fn(int x) { return x * x; }
static inline int max_fn(int a, int b) { return a > b ? a : b; }

int main(void)
{
```

```
    int a = 4, b = 7;
    printf("Rs 12 in paise = %ld\n", RUPEES_TO_PAISE(12));
    printf("SQUARE(a + 1) = %d\n", SQUARE(a + 1));
    printf("MAX_VALUE(a, b) = %d\n", MAX_VALUE(a, b));
    printf("inline square(5) = %d\n", square_fn(5));
    printf("inline max(4, 7) = %d\n", max_fn(a, b));
    return 0;
}
```



The equivalent preprocessor view can be inspected with: `gcc -E program.c`. The compiler sees the expanded expressions rather than the original macro calls.

TASK 5 – Header Files and Conditional Compilation

A normal program is organised into included headers, macro definitions and type declarations, function prototypes, main(), and supporting function definitions. A user header should use include guards:

```
#ifndef BILLING_H
#define BILLING_H
#define FIXED_CHARGE_PAISE 5000L
long compute_bill(int units);
#endif
```

• #include searches the implementation's standard include locations.

• #include "billing.h" searches the current/source include location before implementation include paths.

• #ifdef DEBUG includes diagnostic statements only when DEBUG is defined, for example gcc -DDEBUG.

• A run-time debug flag remains in the executable and tests the flag when executed; conditional compilation can remove debug code entirely from a non-debug build.

DEBUG example

```
#include <stdio.h>
#define FIXED_CHARGE 5000L
#define RATE_PAISE 225L

long simple_bill(int units)
{
    long energy = units > 100 ? (long)(units - 100) * RATE_PAISE : 0;
#ifdef DEBUG
    printf("[DEBUG] units=%d energy=%ld paise\n", units, energy);
#endif
    return FIXED_CHARGE + energy;
}

int main(void)
{
    printf("Bill = Rs %ld.%02ld\n",
           simple_bill(150) / 100, simple_bill(150) % 100);
    return 0;
}
```

```
$ gcc -std=c11 -Wall -Wextra -DDEBUG program.c -o program
$ ./program
[DEBUG] units=150 energy=11250 paise
[DEBUG] units=150 energy=11250 paise
Bill = Rs 162.50
```

TASK 6 – Read and Print Data Correctly

Common printf/scanf specifiers: int %d; unsigned int %u; long %ld; long long %lld; float %f in printf and %f for float* in scanf; double %f in printf and %lf for double* in scanf; char %c; string %s. Always match the conversion specifier to the exact pointer/value type. A mismatched specifier causes undefined behaviour.

Check scanf's return value. For two integer inputs, scanf must return 2. If it returns fewer values, reject the input. For robust line-based input, fgets() reads a bounded line and strtol() validates conversion and trailing characters. Processing a line of length L takes O(L) time and uses O(L) buffer space; it generally recovers from invalid input more cleanly than scanf.

Validated input program

```c
#include <stdio.h>

int main(void)
{
    int previous, current;
    unsigned int units;
    long bill_paise;

    printf("Enter previous and current readings: ");
    if (scanf("%d %d", &previous, &current) != 2 ||
        previous < 0 || previous > 65535 ||
        current < 0 || current > 65535) {
        printf("Invalid input\n");
        return 1;
    }

    if (current < previous && !(previous >= 65000 && current <= 535)) {
        printf("Invalid reading: decrease without valid rollover\n");
        return 1;
    }

    units = current >= previous ? (unsigned)(current - previous)
                                : (unsigned)(65536 - previous + current);
    bill_paise = 5000L;
    if (units > 100) bill_paise += (long)(units - 100) * 225L;
    printf("Units = %u\nBill = Rs %ld.%02ld\n",
           units, bill_paise / 100, bill_paise % 100);
    return 0;
}
```

```
$ gcc -std=c11 -Wall -Wextra program.c -o program
$ ./program
Enter previous and current readings: Units = 330
Bill = Rs 567.50
```

The example execution uses previous reading 1200 and current reading 1530. It validates the reading range, rejects an ordinary decrease, and prints the amount as rupees and paise.

TASK 7 – Control Type Conversion and Precision

• Integer promotion converts small integer types such as unsigned short before arithmetic; the final cast in the rollover function restores 16-bit modular behaviour.

• Integer division discards the fractional part: 7 / 2 is 3, while 7.0 / 2 is 3.50.

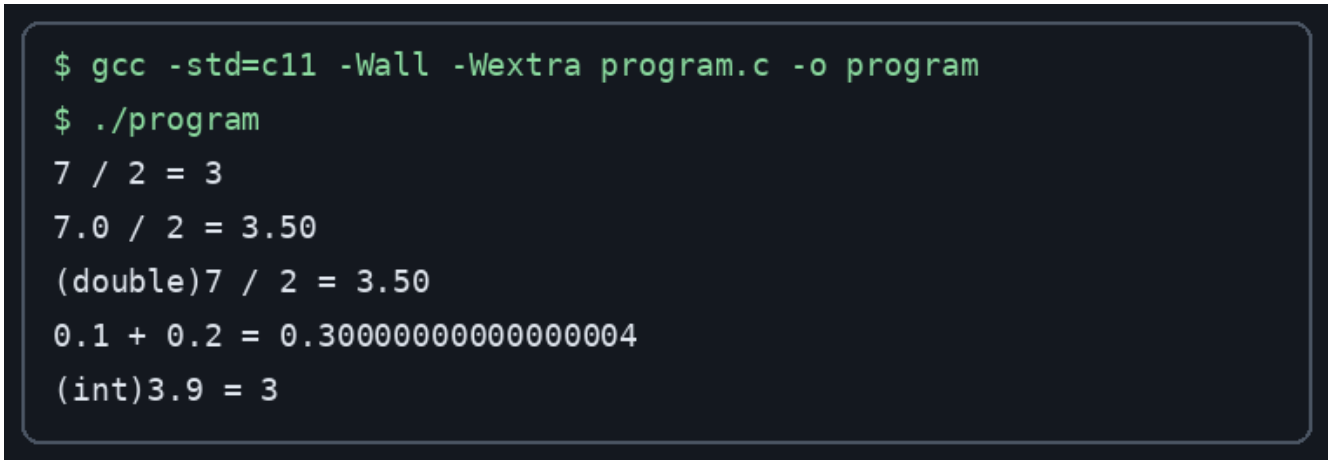
• Casting a large long long to int can lose information or produce an implementation-defined result when the value is not representable.

• When a fractional charge must be converted to paise, define a single rounding rule, such as round-half-up, and apply it at the conversion boundary. Prefer integer rate arithmetic where possible.

Demonstrations

```c
#include <stdio.h>

int main(void)
{
    printf("7 / 2 = %d\n", 7 / 2);
    printf("7.0 / 2 = %.2f\n", 7.0 / 2);
    printf("(double)7 / 2 = %.2f\n", (double)7 / 2);
    printf("0.1 + 0.2 = %.17f\n", 0.1 + 0.2);
    printf("(int)3.9 = %d\n", (int)3.9);
    return 0;
}
```



Unexpected results: 7/2 gives 3 because both operands are integers; 0.1+0.2 may print a value close to, but not exactly, 0.3 because decimal fractions are not generally exact in binary; (int)3.9 gives 3 because conversion to integer truncates toward zero.

If all calculations used float and rounded once at the end, the final error would depend on the representation and number of additions. Each addition is O(1), but integer paise gives exact whole-paisa totals and usually uses 4 or 8 bytes; float commonly uses 4 bytes.

TASK 8 – Trace the Compilation Process

• **Preprocessing:** gcc -E program.c expands macros and includes header contents.

• **Compilation:** gcc -S program.c produces assembly source.

• **Assembly:** gcc -c program.c produces an object file.

• **Linking:** gcc program.o -o program combines object code and required libraries into an executable.

A compile-time error is a syntax/type error such as a missing semicolon. A link-time error occurs when a declared function has no matching definition. A run-time error occurs during execution, such as invalid input or division by zero. gcc -Wall -Wextra helps identify unused variables, suspicious conversions and other potential defects; warnings should be resolved rather than ignored.

For multiple source files, compile each changed .c file to its object file and link all objects. A change to one source file normally requires recompiling only that file, then relinking; a single-file build recompiles the whole source. Separate object files use additional disk space but improve modularity and incremental builds.

Compilation commands

```
gcc -std=c11 -Wall -Wextra -E billing.c -o billing.i
gcc -std=c11 -Wall -Wextra -S billing.c -o billing.s
gcc -std=c11 -Wall -Wextra -c billing.c -o billing.o
gcc billing.o -o billing
```

TASK 9 – Implement and Test the Billing Program

The complete program below uses modular unsigned arithmetic for a single meter rollover and long long paise for billing. It implements the stated slab tariff and tests the key boundaries.

```
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;
#define FIXED_PAISE 5000LL
#define RATE2 225LL
#define RATE3 450LL

unsigned int used_units(meter_t old_read, meter_t new_read)
{
    return (unsigned short)(new_read - old_read);
}

long long bill_paise(unsigned int units)
{
    long long total = FIXED_PAISE;
    if (units > 200) total += (long long)(units - 200) * RATE3;
    if (units > 100) {
        unsigned int middle = units < 200 ? units - 100 : 100;
        total += (long long)middle * RATE2;
    }
    return total;
}

int main(void)
{
    const unsigned int cases[] = {0, 100, 150, 200, 350, 256};
    for (int i = 0; i < 6; ++i) {
        long long p = bill_paise(cases[i]);
        printf("%3u units -> Rs %lld.%02lld\n",
               cases[i], p / 100, p % 100);
    }
    printf("Rollover 65400 -> 120: %u units\n",
           used_units((meter_t)65400, (meter_t)120));
    return 0;
}
```

```
$ gcc -std=c11 -Wall -Wextra program.c -o program
$ ./program
0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
256 units -> Rs 527.00
Rollover 65400 -> 120: 256 units
```

Test table

Test	Units	Expected bill	Observed
Zero units	0	Rs 50.00	Rs 50.00

Test	Units	Expected bill	Observed
Exactly 100	100	Rs 50.00	Rs 50.00
150 units	150	Rs 162.50	Rs 162.50
Exactly 200	200	Rs 275.00	Rs 275.00
350 units	350	Rs 950.00	Rs 950.00
Rollover 65400→120	256	256 units	256 units

Five-bill hand calculation: choose 0, 100, 150, 200 and 350 units. Their bills are Rs 50.00 + Rs 50.00 + Rs 162.50 + Rs 275.00 + Rs 950.00 = **Rs 1,487.50**. Invalid letters, negative readings and decreases without a valid rollover must be rejected by the input-validation layer.

An array-driven tariff loop is more flexible when the number of slabs s changes. It takes O(s) time per bill and O(s) storage for tariff data. Fixed if statements have constant time for a small fixed tariff and may be simpler, while arrays are easier to extend.

TASK 10 – Compare Expectations with Execution

For an illustrative bill of Rs 950.00 (95,000 paise), the district totals are:

```
#include <stdio.h>

int main(void)
{
    const long long bill_paise = 95000LL; /* example: Rs 950.00 */
    const int counts[] = {4000, 60000, 800000};
    for (int i = 0; i < 3; ++i) {
        long long total = bill_paise * counts[i];
        double total_double = 950.00 * counts[i];
        printf("%d bills: integer=%lld paise, double=%.2f rupees\n",
               counts[i], total, total_double);
    }
    printf("sizeof(int)=%zu long=%zu long long=%zu double=%zu\n",
           sizeof(int), sizeof(long), sizeof(long long), sizeof(double));
    return 0;
}
```

```
$ gcc -std=c11 -Wall -Wextra program.c -o program
$ ./program
4000 bills: integer=380000000 paise, double=3800000.00 rupees
60000 bills: integer=5700000000 paise, double=57000000.00 rupees
800000 bills: integer=76000000000 paise, double=760000000.00 rupees
sizeof(int)=4 long=8 long long=8 double=8
```

Households	Total in paise	Total in rupees
4,000	380,000,000	Rs 3,800,000.00
60,000	5,700,000,000	Rs 57,000,000.00
800,000	76,000,000,000	Rs 760,000,000.00

An ordinary 32-bit signed int cannot hold 76,000,000,000 paise. A 32-bit long also cannot hold it, but a 64-bit long or long long can. C guarantees long long to have at least 64 bits. double can represent these totals at this scale, but it does not guarantee exact decimal paise for every possible value. Integer paise is the safer accounting representation.

Time per addition is O(1) for int, long, long long and double. Typical storage is 4 bytes for int, 4 or 8 bytes for long depending on platform, 8 bytes for long long, and commonly 8 bytes for double. The actual sizes must be verified with sizeof.

TASK 11 – Recommend and Defend the Final Design

• Store each meter reading as an unsigned 16-bit type only when the meter specification confirms a 16-bit register. Calculate consumption with modular arithmetic and validate that at most one rollover is possible.

• Store money as integer paise in long long for household and district totals.

• Keep slab limits and rates in one named configuration section. Use #define for compile-time limits and typed const variables for values needing scope and type checking.

• Use include guards, clear function prototypes, input validation and optional #ifdef DEBUG diagnostics.

• The design scales to 800,000 households when totals use 64-bit integer storage and input processing is performed record by record rather than storing every bill in memory.

Limitations: two readings cannot reveal multiple meter rollovers; the tariff example assumes one fixed tariff period; tax, subsidies, meter faults and category-specific rules would need additional requirements. A configuration file is preferable when tariff changes must not require recompilation. Reading t tariff values takes O(t) startup time and O(t) memory (or O(1) extra memory if processed as a small fixed table); it avoids rebuilding and redistributing the program for every tariff revision, but the file must be validated.

TASK 12 – Re-evaluate and Adapt the Design

A. Impact Analysis

• **Changed data:** the system now receives watt-hour readings every 15 minutes, creating 35,040 records per household per year. Time-of-day classification is required, and peak-period energy from 6 PM to 10 PM costs 20% more.

• **Data types:** a 16-bit cumulative register is no longer sufficient for yearly energy totals. Use a 64-bit unsigned integer for watt-hours or cumulative energy, a suitable timestamp type for each interval, and 64-bit integer paise for monetary totals.

• **Tariff constants:** rates and peak multiplier should be loaded from a validated configuration file or database at startup rather than hard-coded only with #define, because the board wants revisions without recompilation.

• **Calculation:** retain integer-based energy and money arithmetic, but split energy into normal and peak-period watt-hours before applying rates. Apply a documented rounding rule when converting watt-hours to paise.

• **Storage and processing:** 4,000 households generate 140,160,000 interval records per year; 60,000 generate 2,102,400,000; 800,000 generate 28,032,000,000. The system should use a database or streaming/batch pipeline, indexed by meter and timestamp, rather than keeping all records in RAM.

Illustrative smart-meter calculation

```
#include <stdio.h>

#define PEAK_NUMERATOR 120
#define RATE_PAISE_PER_KWH 225LL

long long charge_paise(unsigned long long watt_hours, int peak_wh)
{
    /* Store energy in Wh; apply the 20% peak premium to peak-period Wh. */
    long long normal = (long long)(watt_hours - (unsigned long long)peak_wh)
                       * RATE_PAISE_PER_KWH / 1000;
    long long peak = (long long)peak_wh * RATE_PAISE_PER_KWH
                     * PEAK_NUMERATOR / 100000;
    return normal + peak;
}

int main(void)
{
    unsigned long long total_wh = 2000;
    int peak_wh = 500;
    long long p = charge_paise(total_wh, peak_wh);
    printf("Total energy = %llu Wh\nPeak energy = %d Wh\n", total_wh, peak_wh);
    printf("Energy charge = %lld paise (Rs %lld.%02lld)\n",
```

```
            p, p / 100, p % 100);
    return 0;
}
```

```
$ gcc -std=c11 -Wall -Wextra program.c -o program
$ ./program
Total energy = 2000 Wh
Peak energy = 500 Wh
Energy charge = 472 paise (Rs 4.72)
```

The example assumes a base rate of 225 paise per kWh and applies a 20% premium to the peak-period portion. For a production system, use 64-bit integers throughout and avoid intermediate truncation by carrying sufficient precision (for example, watt-hours × paise/kWh) until the final paise rounding step.

Final Conclusion

The recommended original design uses unsigned meter arithmetic for rollover, integer paise for exact bills, named tariff constants, validated input, modular functions and 64-bit totals. For the smart-meter requirement, modify the design: use 64-bit watt-hour counters, timestamps, separate peak/off-peak energy accounting, external tariff configuration and scalable persistent storage. This preserves exact financial arithmetic while meeting the new time-of-day and tariff-update requirements.

Submission Checklist

• This is one combined PDF containing the written solutions, source code and output captures.

• Review the register number and name on the cover page before submission.

• Upload this same final PDF to both the required GitHub repository and the SIMATS submission portal.

• Confirm that both platforms show the latest uploaded version; upload to those platforms manually unless access and permission for direct upload are provided.